Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor: Dr. Ahmed Raza	**Date:** 19-February-2026
Day: Thursday	**Semester:** 6th

Lab 2: The Geometry of Overfitting - Polynomial Regression & Ridge

Lab Title	CLO	BT	PLO	Weightage
The Geometry of Overfitting: Polynomial Regression & Ridge				

Name	Roll Number	Report /10	Viva /5	Total /15
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab, the student will be able to:

- Construct high-degree polynomial feature matrices.
- Implement the Normal Equation using matrix inversion.
- Observe numerical instability and coefficient explosion.
- Implement Ridge Regression from scratch.
- Analyze how regularization stabilizes learning.

2 Equipment Required

- PC with Python 3.x installed (Anaconda recommended).
- IDE (Jupyter Notebook, Spyder, VS Code, or PyCharm).
- NumPy, Pandas, and Matplotlib libraries.

3 Theory and Background

High-capacity models can perfectly fit small datasets but often generalize poorly. Polynomial regression of high degree increases model flexibility but may lead to overfitting and numerical instability due to ill-conditioned matrices.

Ridge Regression introduces L2 regularization to control model complexity:

$$w = (X^T X + \lambda I)^{-1} X^T y \tag{1}$$

This shifts eigenvalues of $X^T X$, stabilizes inversion, and reduces coefficient magnitude.

4 Part A — The Overfitting Demo

Dataset: $y = \sin(2\pi x) + \epsilon$ (10 data points, fixed random seed).

Task A1: Polynomial Feature Matrix (2 Marks)

Implementation Box

Generate polynomial features up to degree 20 (include bias term). Verify matrix dimensions.

Code:

```
def build_polynomial_features(x, degree):
    # TODO: build Vandermonde-style matrix including bias term

    # Converting the input into a column shape with one colomn and it's rows equal to the number of samples.
    x = np.array(x).reshape(-1, 1)

    # Starting with a column of 1s (this helps the model learn the intercept)
    features = np.ones((x.shape[0], 1))

    # Adding columns with the help of loop and hstack till x, x², x³ ... up to max_degree
    for i in range(1, degree + 1):
        features = np.hstack((features, x ** i))

    return features
```

Verification of matrix Dimensions:

```
xTrain, yTrain = generate_dataset(n=10, noise_std=0.1, seed=42)

degree = 20

# Task A1 --> Build polynomial feature matrix (bias + up to degree 20)
XTrainPoly = build_polynomial_features(xTrain, degree)

# Task A1 --> Verify dimensions
print("\nTask A1 --> Dimension check")
print(f"xTrain: {xTrain.shape} | yTrain: {yTrain.shape} | XTrainPoly: {XTrainPoly.shape}") #XTrainPoly = 10 * 21 (10 rows, 21 columns: bias + 20 degrees)
```

Output:

```
Task A1 --> Dimension check
xTrain: (10, 1) | yTrain: (10, 1) | XTrainPoly: (10, 21)
```

Task A2: Normal Equation Regression (3 Marks)

Implementation Box

Implement $w = (X^T X)^{-1} X^T y$.
Fit a 20-degree polynomial to 10 data points.
Plot training data and fitted curve (100 test points).

1) Code:

```python
def normal_equation(X, y):
    # Task A2 --> Implement Normal Equation solution: (X^T X) w = X^T y.
    xTx = X.T @ X
    xTy = X.T @ y
    weights = np.linalg.solve(xTx, xTy)

    return weights.reshape(-1, 1)
```

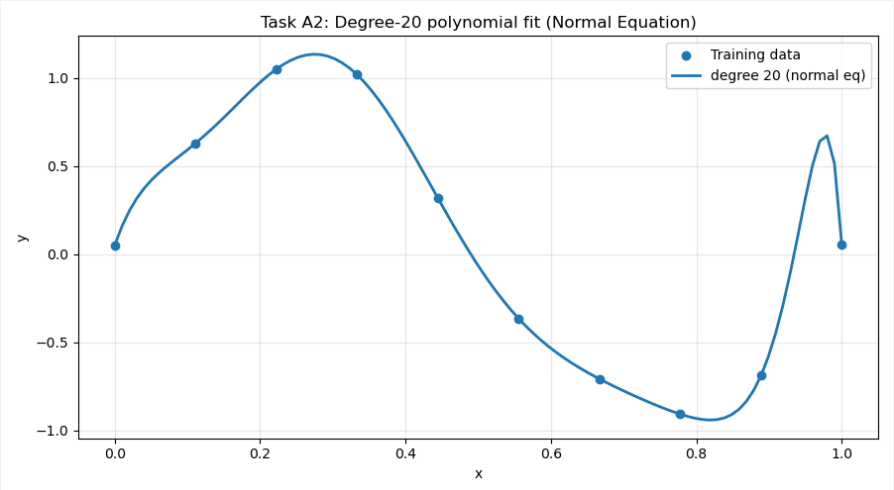
2) Code:

```python
# Task A2 --> Fit 20-degree polynomial using the normal equation
wNE = normal_equation(XTrainPoly, yTrain)

# Task A2 --> Plot fitted curve using exactly 100 test points
xPlot = np.linspace(0.0, 1.0, 100).reshape(-1, 1)
XPlotPoly = build_polynomial_features(xPlot, degree)
yPredNE = XPlotPoly @ wNE

# Task A2 --> Plot ONLY the normal-equation fit
plot_results(
    x_train=xTrain,
    y_train=yTrain,
    x_plot=xPlot,
    pred_curves=[yPredNE],
    labels=[f"degree {degree} (normal eq)"],
    title="Task A2: Degree-20 polynomial fit (Normal Equation)"
)
```

2) Plot



Task A3: Coefficient Explosion & Stability (2 Marks)

Analysis Box

Print coefficient magnitudes.
Compute condition number of ($X^T X$).
Explain numerical instability.

1+2)
Output (Values Printed):

```
Task A3 --> Unregularized stats
Condition number (X^T X): 4.9471e+17
Weight size (L2 norm)   : 15130.6
Largest weight (abs)    : 7311.54
First 10 weights        : [ 4.96714192e-02  1.20634043e+01 -1.33010759e+02  9.03034705e+02
 -2.56405533e+03  1.45009346e+03  5.36903932e+03 -7.31153627e+03
 -2.40553470e+03  5.97084258e+03]
```

3)

Explanation for Instability

- With a 20-degree polynomial, we created many features (1, x, x^2, …, x^{20}) from only 10 data points, so many feature columns ended up very similar to each other.
- Because of that, the matrix X^TX became ill-conditioned (almost non-invertible).
- When the condition number was huge, tiny noise in the data caused big changes in the solved weights.
- That is why the coefficients became extremely large (coefficient "explosion") and the fitted curve became unstable.
- So, the instability came from solving the normal equation using an ill-conditioned X^TX, which happened because we used a very flexible (degree-20) model on only 10 noisy data points.
- In eigenvalue terms: very small eigenvalues of $X^T X$ make the normal-equation solution hard to compute accurately (ill-conditioned), and without regularization ($\lambda = 0$) nothing shifts those eigenvalues up, so the instability and coefficient explosion get worse

5 Part B — Ridge Regression

Task B1: Ridge Closed-Form Implementation (3 Marks)

Implementation Box

Implement $w = (X^T X + \lambda I)^{-1} X^T y$.
Do NOT regularize the bias term.
Use NumPy only (no sklearn).

Code (no sklearn is used):

```python
def ridge_regression(X, y, lambda_):
    numberOfFeatures = X.shape[1]

    xTx = X.T @ X
    xTy = X.T @ y

    # Task B1 --> Ridge closed-form solve, but do NOT regularize the bias term.
    regMatrix = float(lambda_) * np.eye(numberOfFeatures, dtype=float)
    regMatrix[0, 0] = 0.0       # bias is column 0

    weights = np.linalg.solve(xTx + regMatrix, xTy)
    return weights.reshape(-1, 1)
```

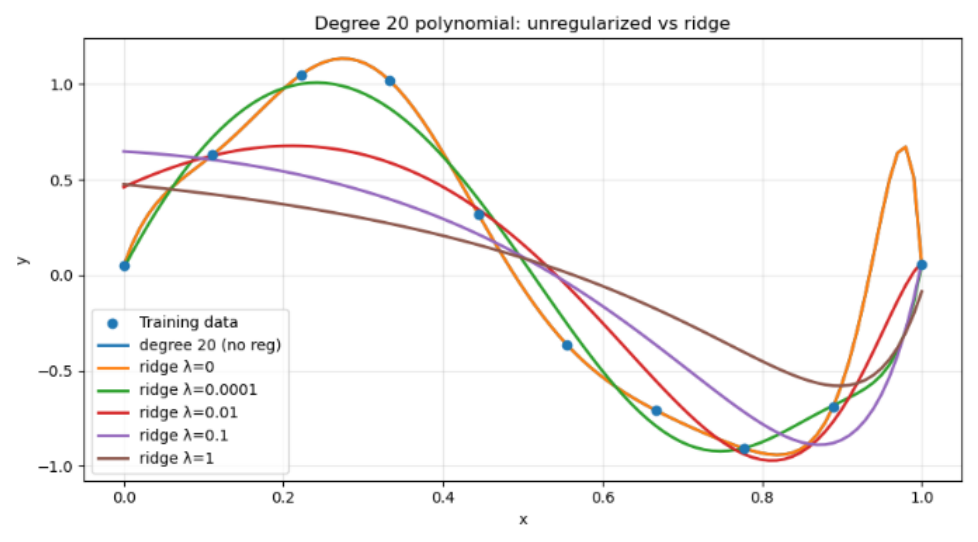
Task B2: Regularization Study (3 Marks)

Test $\lambda \in \{0, 10^{-4}, 10^{-2}, 10^{-1}, 1\}$.

Results Box

For each λ:
Plot fitted curve.
Record coefficient norm.
Record condition number.

1) Plot



2+3)

```
Task B2 --> Ridge results

    lambda | weight norm | condition number

         0 |  15130.6397 |       4.9471e+17

    0.0001 |     22.1760 |       2.8073e+05

      0.01 |      5.6970 |       2.8081e+03

       0.1 |      1.7918 |       2.8155e+02

         1 |      0.9644 |       2.8893e+01
PS D:\ML-Labs\Lab02>
```

Task B3: Stability Analysis (2 Marks)

Analysis Box

Compare condition numbers before and after regularization.
Explain eigenvalue shift intuition.
Explain geometric meaning of Ridge Regression.

Answer#1:

Comparison Before and after regularization

Before ridge ($\lambda = 0$) the condition number was extremely large: **4.9471×10^{17}** and the weight norm was **15130.6397**. This shows the solution was unstable, tiny noise in the data could cause very large coefficients. After adding ridge regularization, the condition number and the weight size drop a lot as λ grows:

- **$\lambda = 0.0001 \rightarrow$ cond = 2.8073×10^5, $\|w\| = 22.1760$**
- **$\lambda = 0.01 \rightarrow$ cond = 2.8081×10^3, $\|w\| = 5.6970$**
- **$\lambda = 0.1 \rightarrow$ cond = 2.8155×10^2, $\|w\| = 1.7918$**
- **$\lambda = 1 \rightarrow$ cond = 2.8893×10^1, $\|w\| = 0.9644$**

So regularization makes the matrix inversion much more stable and prevents the coefficients from exploding. As λ increases the solution becomes smoother and more reliable.

Answer#2:

Eigenvalue-shift intuition

When the matrix we need to invert has some very small numbers (in certain directions), the inverse becomes extremely sensitive — tiny changes in data can give huge changes in the solution. Adding λ on the diagonal is like increasing those tiny numbers so they are no longer near zero. This "shifts up" the small directions and makes the inverse less sensitive. In short: ridge adds a small positive amount to every direction so the problem stops being dominated by tiny, unstable directions.

Answer#3:

Geometric meaning of Ridge Regression

Ridge regression adds an L2 penalty to the training loss. This penalty keeps the weights small, so the model becomes smoother and more stable, and it reduces overfitting. If we imagine this visually, the possible values of weights are limited inside a circular (or spherical) boundary around zero. The final solution is chosen from inside this boundary. Because of this restriction, the weights become smaller compared to normal regression.

So basically, ridge pulls the weights closer to zero to make the model more stable. However, it does not make them exactly zero.

6 Reflection (3 Marks)

Reflection Box

1. Why does a 20-degree polynomial overfit 10 points?
2. What happens when $\lambda \rightarrow 0$?
3. What happens when λ becomes large?
4. Does Ridge increase bias or reduce variance?

Answer#1:

A 20-degree polynomial overfit 10 points

Because a degree-20 polynomial has too many free parameters compared to only 10 training points, so it can "bend" a lot to match almost every point.

This makes the curve follow noise in the data, so it looks perfect on training points but becomes wiggly and unreliable between points (poor generalization).

Answer#2:

What happens when $\lambda \rightarrow 0$?

When λ goes to 0, ridge regression becomes the same as normal regression (no regularization).

So weights can become very large and the solution becomes unstable again, like what we saw at $\lambda = 0$ where the condition number was 4.9471×10^{17} and the weight norm was 15130.6397.

Answer#3:

What happens when λ becomes large?

When λ is large, ridge puts a strong penalty on large weights, so the weights shrink a lot and the curve becomes smoother.

In our results, when $\lambda = 1$, the weight norm dropped to 0.9644 and the condition number dropped to 2.8893×10^{1}, which shows the model became much more stable.

If λ is too large, the model can underfit (it becomes too simple and doesn't follow the pattern well).

Answer#4:

Does Ridge increase bias or reduce variance?

Ridge usually reduces variance because the model becomes less sensitive to small changes in the training data (weights don't blow up).

But it can slightly increase bias because we restrict the weights, so training fit can get worse compared to no-regularization

7 Conclusion (1 Mark)

Conclusion Box

Write a brief summary of what you learned from this lab, focusing on capacity control and regularization in machine learning.

- In this lab, I learned that a high-degree polynomial can easily overfit a small dataset and can also become numerically unstable.
  I implemented polynomial regression using the normal equation and saw very large weights and a very high condition number.
  Then I implemented Ridge Regression and learned that adding regularization keeps weights small and makes the solution more stable.
  Overall, regularization is important for capacity control and for reducing overfitting in machine learning

Assessment Rubrics

Method: Lab reports and instructor observation during lab sessions

Outcome Assessed:

1. Ability to conduct experiments, as well as to analyze and interpret data (P).
2. Ability to use the techniques, skills, and modern engineering tools necessary for engineering practice (P).

Performance	Exceeds expectation (4-5)	Meets expectation (3-2)	Does not meet expectation (1)	Marks
1. Realization of Experiment [a, b]	Selects relevant equipment to the experiment, develops setup diagrams of equipment connections or wiring.	Needs guidance to select relevant equipment and to develop equipment connection or wiring diagrams.	Incapable of selecting relevant equipment to conduct the experiment.	
2. Conducting Experiment [a, b]	Does proper calibration of equipment, carefully examines equipment moving parts, and ensures smooth operation.	Calibrates equipment, examines equipment moving parts, and operates the equipment with minor error.	Unable to calibrate appropriate equipment, and equipment operation is substantially wrong.	
3. Laboratory Safety Rules [a]	Respectfully and carefully observes safety rules and procedures.	Observes safety rules and procedures with minor deviation.	Disregards safety rules and procedures.	
6. Data Collection [a]	Plans data collection to achieve experimental objectives, and conducts an orderly and complete data collection.	Plans data collection to achieve experimental objectives, and collects complete data with minor error.	Does not know how to plan data collection; data collected is incomplete and contain errors.	
7. Data Analysis [a]	Accurately conducts simple computations and statistical analysis; correlates results to known theoretical values; accounts for measurement errors.	Conducts simple computations with minor error; reasonably correlates results to known theoretical values; attempts to account for errors.	Unable to conduct simple statistical analysis; no attempt to correlate or explain errors.	
8. Computer Use [a]	Uses computer to collect and analyze data effectively.	Uses computer to collect and analyze data with minor error.	Does not know how to use computer to collect and analyze data.	
			Total	

Faculty:

Name: _____

Signature: _____

Date: _____